Object Oriented Programming

ICT2122

Introduction to Design Patterns

P.H.P. Nuwan Laksiri
Department of ICT
Faculty of Technology
University of Ruhuna.

Lesson 09

Recap – Java Threads

- Introduction to Java threads
- Multithreading in Java
- Threads vs Processes
- Life cycle of a Thread
- Creating Threads
  - By extending
  - By implementing

What we discuss Today ……..

- Introduction
- Design Pattern Space
- Object Oriented Design Patterns
  - Singleton

Introduction

- Why
  - Design patterns make it easier to **reuse** successful **Designs** and **Architectures**
- **Design Pattern → Right Design** (Faster)
- Make Novice an Expert(Through best practices)

- "Each pattern describe a problem which occurs over and over again in our environment, and then describe the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice"

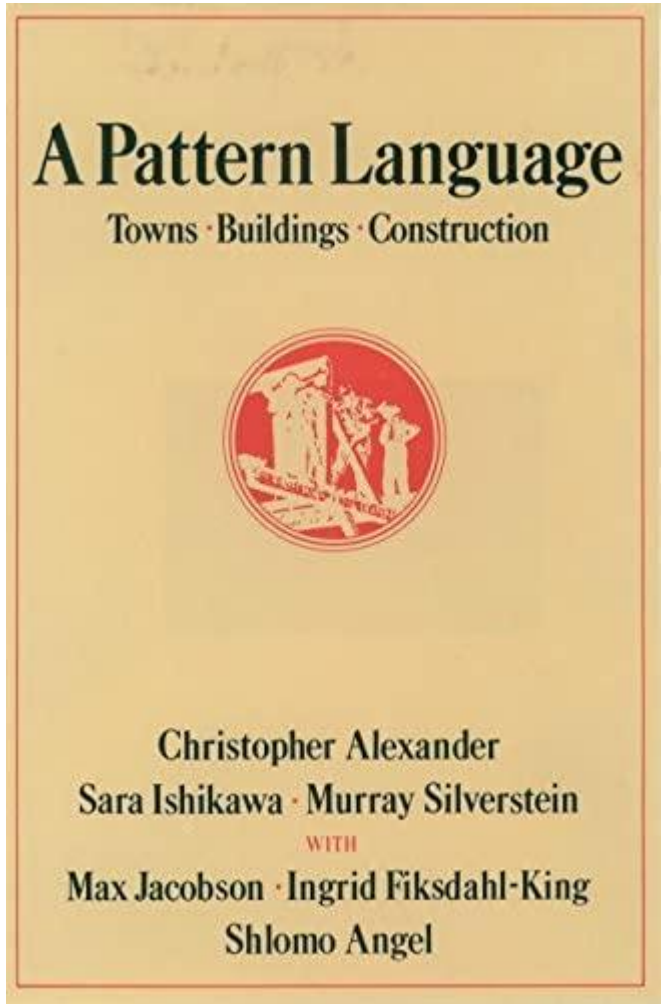
[Christopher Alexander]

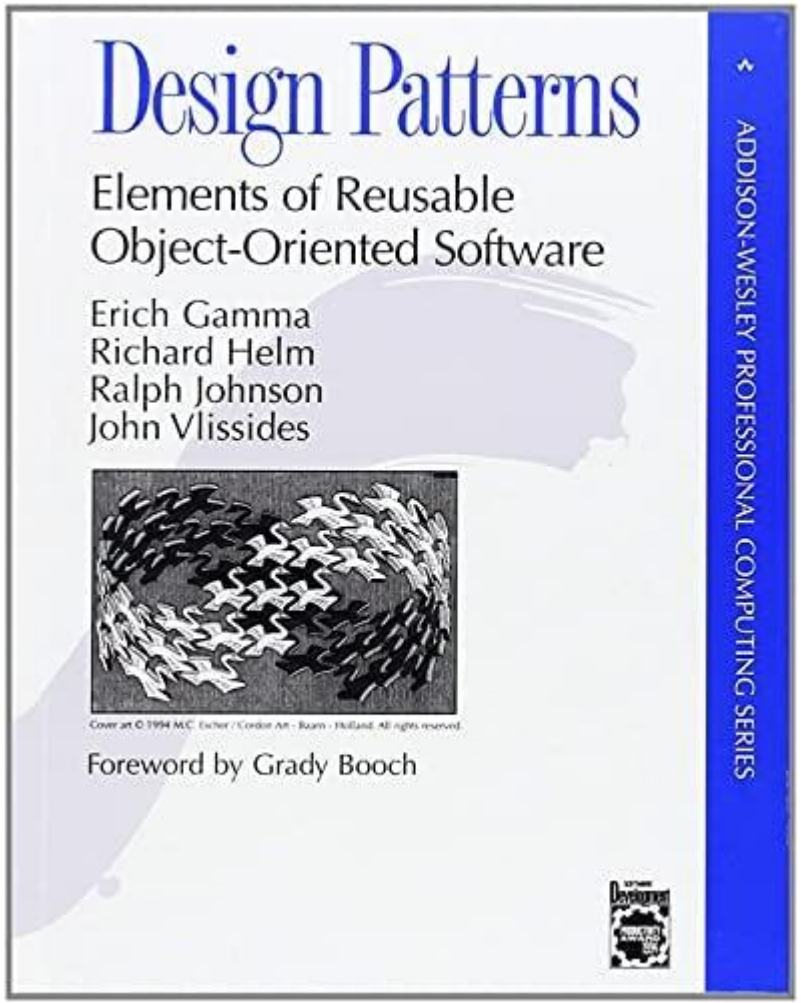
Introduction

- Design pattern capture the end product of the design process in terms of
  - Relationship among Classes and Objects
  - Recording decisions for the design
  - Alternatives for the design
  - Trade-offs that lead to the design
  - Concrete example

History of Design Patterns

- Who invented patterns?





Gang of Four (GOF)?



John Vlissides



Erich Gamma



Ralph Johnson



Richard Helm



Program to an interface not an implementation

Favor object composition over inheritance

Design Pattern Space

		Purpose		
		Creational	Structural	Behavioral
Scope	Class	**Factory Method**	**Adapter**	**Interpreter** **Template Method**
	Object	**Abstract Factory** **Builder** **Prototype Singleton**	**Adapter** **Bridge** **Composite** **Decorator** **Facade** **Proxy**	**Chain of Responsibility** **Command** **Iterator** **Mediator** **Memento** **Flyweight** **Observer** **State** **Strategy** **Visitor**

Design Pattern Space

- Most patterns are in the object scope

- Creational Patterns
  - These design patterns provide a way to create objects while hiding the creation logic, rather than instantiating objects directly using new operator.
  - This gives program more flexibility in deciding which objects need to be created for a given use case.

- Structural Patterns
  - These design patterns concern class and object composition. Concept of inheritance is used to compose interfaces and define ways to compose objects to obtain new functionalities.

- Behavioral Patterns
  - These design patterns are specifically concerned with communication between objects.

Design Pattern Space

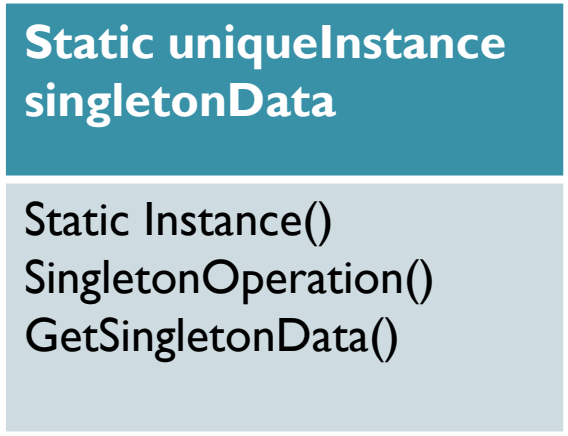
- Two Criteria
  - Purpose – What pattern does
    - Creational – Process of Object creation
    - Structural – Composition of Classes or Objects
    - Behavioral – Classes or Objects interact and distribute responsibility
  - Scope – Whether the pattern applied primarily to a Class or and Object
    - Class – Deals with relationships between classes and sub-classes (Via inheritance and static)
    - Object – Deals with object relationships (can be changed at runtime and dynamic)

What do you think?

- Imagine a system where every part of the program creates its own database connection object.
  - What problems could happen?"
  - What if we allowed only ONE shared object for the entire application?"

Creational Design Patterns

- Singleton
  - Is used when only one instance of an object is needed throughout the lifetime of an application
  - How
    - Make the class responsible for keeping track of its sole instance (users cannot explicitly instantiate this class)
    - Provide a global way to access the instance
  - Client access singleton instance only via SingletonOperation()

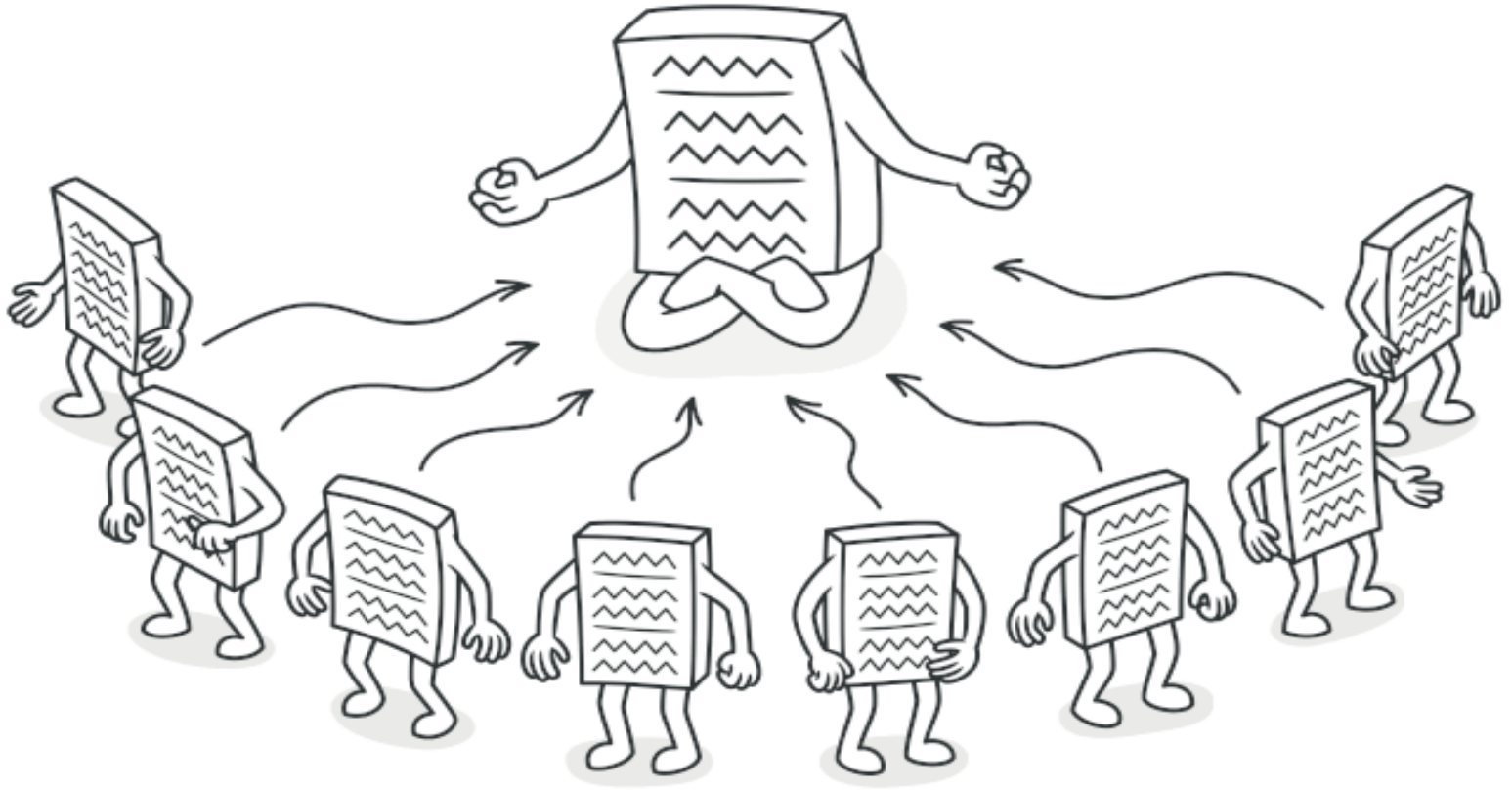


Creational Design Patterns

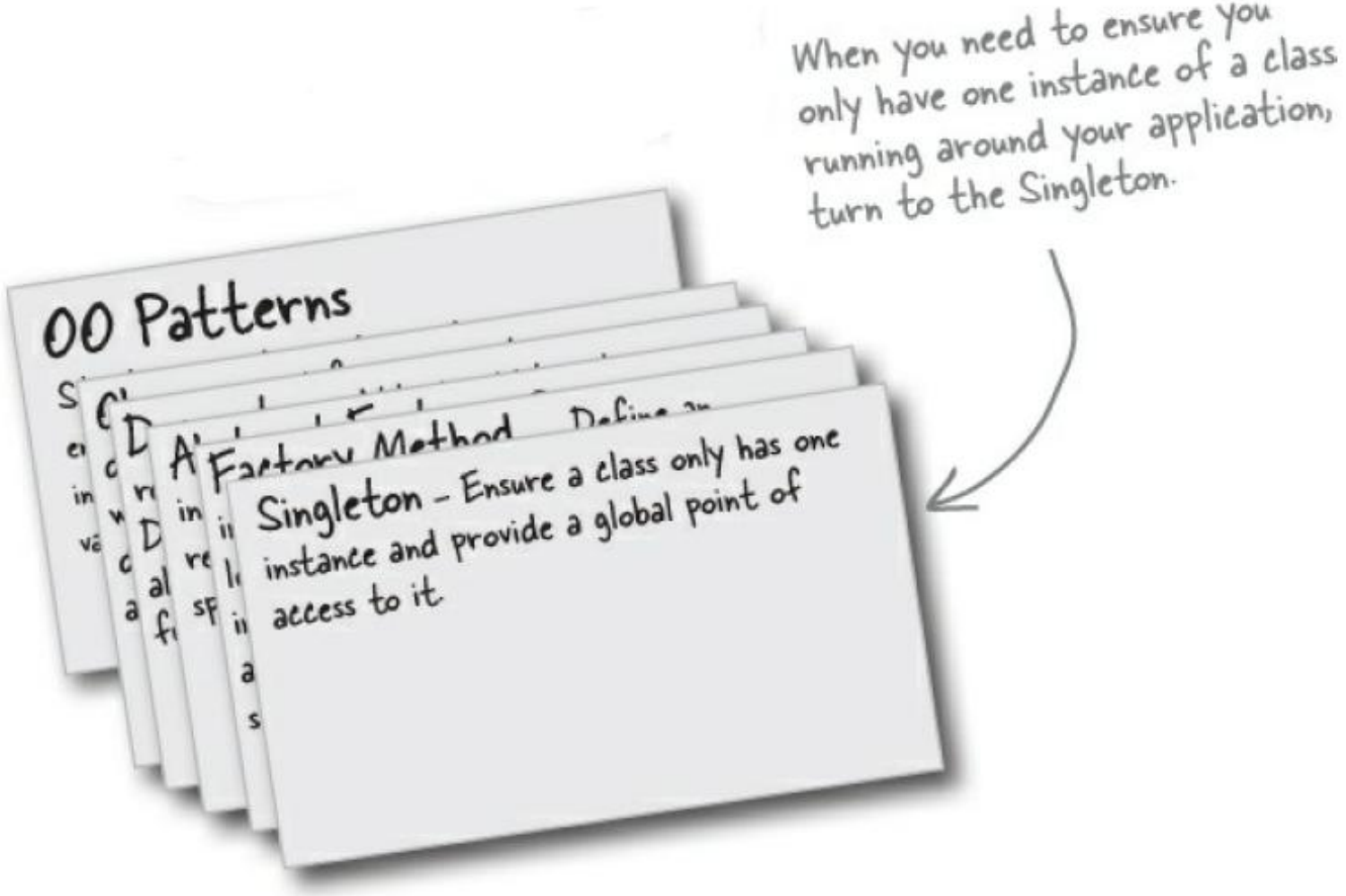
- Singleton Design Pattern
  - Example
    - Clip Board
    - Windows Registry (System Parameters)

Singleton Design Pattern

Singleton is a creational design pattern that lets you ensure that a class has only one instance, while providing a global access point to this instance.



Singleton Design Pattern



Singleton Design Pattern

- The Singleton pattern solves two problems at the same time
  - Ensure that a class has just a single instance
  - Provide a global access point to that instance

Singleton Design Pattern

- Ensure that a class has just a single instance
  - Why would anyone want to control how many instances a class has?
    - The most common reason for this is to control access to some shared resource
      - ex: a database or a file

Singleton Design Pattern

- Provide a global access point to that instance
  - Remember the global variables that you used to store some essential objects?
    - While they're very handy, they're also very unsafe since any code can potentially overwrite the contents of those variables and crash your application
  - Just like a global variable, the Singleton pattern lets you access some object from anywhere in the program
    - However, it also protects that instance from being overwritten by other code

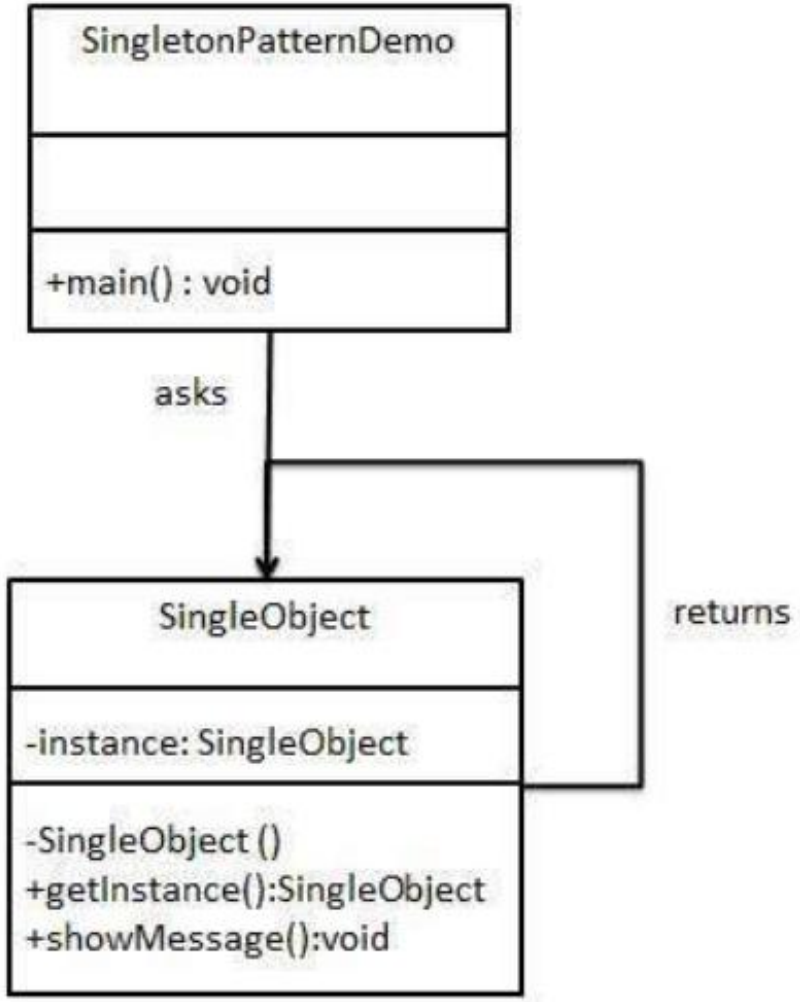
Singleton Design Pattern

- How to implement
  - Make the constructor of the class private
  - Add a private static field to the class for storing the singleton instance
  - Declare a public static creation method for getting the singleton instance

Singleton Design Pattern

- Advantages
  - You can be sure that a class has only a single instance
  - You gain a global access point to that instance
  - The singleton object is initialized only when it's requested for the first time
- Disadvantages
  - Violates the Single Responsibility Principle
  - The Singleton pattern can mask bad design
  - The pattern requires special treatment in a multithreaded environment
  - It may be difficult to unit test the client code of the Singleton

Singleton Design Pattern - Example



Hands On

DB : student

Table : basicdata

stu_id	stu_name	stu_address
TG/001	Amila	Matara
TG/002	Basil	Hambantota
TG/003	Charles	Jaffna
TG/004	Debian	Colombo

Table : marks

stu_id	chemistry	physics	maths
TG/001	65	75	80
TG/002	50	40	60
TG/003	55	85	35
TG/004	85	75	85

Student ID	Name	Chemistry Grade	Physics Grade	Maths Grade	Overall Grade

- Grade references

Range	Grade
0-34	F
35-64	C
65-84	B
85-100	A

Why Should We Learn Design Patterns?

- ???



Summary

- Introduction
- Design Pattern Space
- Object Oriented Design Patterns
  - Singleton
    - What
    - How to implement
    - Advantages and Disadvantages
    - Example

References

- How To Program (Early Objects)
  - 10th Edition
  - By H .Deitel and P. Deitel
- Head First Java
  - 2nd Edition
  - By Kathy Sierra and Bert Bates
- Head First Design Patterns
  - 2nd Edition
  - By Eric Freeman, Elisabeth Robson, Bert Bates, Kathy Sierra

Questions ???



Thank You